

Simultaneous-dial specification for the version 2 connection layer.

The draft's whole rule for duplicate connections is one sentence ("Connection Management"): "A node SHOULD maintain at most one connection to a given remote address." It says nothing about the simultaneous-open race — two nodes dialling each other at once, so that two connections exist before either side can see the duplicate — nor which of the two connections a node should keep. This module checks what each way of filling that gap does.

Two peers each want a connection to the other. There are two possible connections, named by their dialer. Each side sees a connection's state only locally; opens and closes propagate asynchronously, so a node can accept an inbound connection while its own outbound dial is still unanswered — the race. A node that observes a duplicate at accept time applies one policy:

"prefer_outbound" refuse the inbound duplicate, keep the own dial. Locally this is also "keep the first connection": the node's own dial always precedes the accept, or the dial would have been blocked.

"prefer_inbound" accept the inbound duplicate, close the own dial. "tiebreak" keep the connection dialled by the fixed *Lower* peer,

whichever side of it this node is on — the one asymmetric convention.

Both symmetric policies are self-defeating when applied by both peers: each node keeps the connection the other node closes, so both connections die and both nodes redial — forever, under adversarial timing. The liveness property *EventuallyOneConnection* is violated. Only the asymmetric tie-break makes the peers agree on a survivor, and with it the property holds.

The model assumes the best case for the draft: each node can even RECOGNIZE the inbound duplicate (in practice an inbound connection arrives from an ephemeral *UDP* port, not the remote's canonical listen address, so matching it to an outbound dial already needs an address-book lookup by *IP*). The gap exists even in this best case.

See `../documents/v2-modeling.md` for the full write-up.

EXTENDS *Naturals*, *FiniteSets*

CONSTANT *Peers* exactly two peers, each wanting a connection to the other
 CONSTANT *Lower* the peer whose dial survives under the tie-break policy
 CONSTANT *Policy* "prefer_outbound" | "prefer_inbound" | "tiebreak"

ASSUME $\text{Cardinality}(\text{Peers}) = 2$

ASSUME $\text{Lower} \in \text{Peers}$

ASSUME $\text{Policy} \in \{\text{"prefer_outbound"}, \text{"prefer_inbound"}, \text{"tiebreak"}\}$

$\text{Other}(p) \triangleq \text{CHOOSE } q \in \text{Peers} : q \neq p$

One record per possible connection, named by its dialer *i*:

d the dialer's local state of it
a the acceptor's local state of it
syn the open is in flight toward the acceptor
fin_d a close notice is in flight toward the dialer
fin_a a close notice is in flight toward the acceptor

$\text{ConnState} \triangleq \{\text{"none"}, \text{"open"}, \text{"closed"}\}$

VARIABLE *conns* [*Peers* → [*d*, *a* : *ConnState*, *syn*, *fin_d*, *fin_a* : BOOLEAN]]

$vars \triangleq \langle conns \rangle$

$NullConn \triangleq [d \mapsto \text{"none"}, a \mapsto \text{"none"},$
 $syn \mapsto \text{FALSE}, fin_d \mapsto \text{FALSE}, fin_a \mapsto \text{FALSE}]$

p 's local view: the connections p currently considers open. p is the dialer of $conns[p]$ and the acceptor of the other connection.

$ViewOpen(p) \triangleq \{i \in Peers : \text{IF } i = p \text{ THEN } conns[i].d = \text{"open"}$
 $\text{ELSE } conns[i].a = \text{"open"}\}$

$Init \triangleq conns = [i \in Peers \mapsto NullConn]$

p dials its peer: only when p sees no connection to it, open or its own earlier dial ("at most one connection to a given remote address").

$Dial \triangleq$
 $\exists p \in Peers :$
 $\wedge conns[p].d = \text{"none"}$
 $\wedge ViewOpen(p) = \{\}$
 $\wedge conns' = [conns \text{ EXCEPT } ![p].d = \text{"open"}, ![p].syn = \text{TRUE}]$

The acceptor q of connection i observes the incoming open. If q 's own dial is still open from its point of view, this is the simultaneous-open race and the policy decides which connection survives; otherwise q accepts. Refusing sets the acceptor side closed and sends a close notice to the dialer; preferring the inbound also closes q 's own dial.

$AcceptConn(i) \triangleq$
 $\text{LET } q \triangleq Other(i)$
 $dup \triangleq conns[q].d = \text{"open"}$
 $keepInbound \triangleq \vee Policy = \text{"prefer_inbound"}$
 $\vee Policy = \text{"tiebreak"} \wedge i = Lower$
 IN
 $\wedge conns[i].syn$
 $\wedge conns[i].a = \text{"none"}$
 $\wedge \vee \wedge \neg dup$
 $\wedge conns' = [conns \text{ EXCEPT } ![i].a = \text{"open"}, ![i].syn = \text{FALSE}]$
 $\vee \wedge dup$
 $\wedge \vee \wedge keepInbound$
 $\wedge conns' = [conns \text{ EXCEPT}$
 $![i].a = \text{"open"}, ![i].syn = \text{FALSE},$
 $![q].d = \text{"closed"}, ![q].fin_a = \text{TRUE}]$
 $\vee \wedge \neg keepInbound$
 $\wedge conns' = [conns \text{ EXCEPT}$
 $![i].a = \text{"closed"}, ![i].syn = \text{FALSE},$
 $![i].fin_d = \text{TRUE}]$

$Accept \triangleq \exists i \in Peers : AcceptConn(i)$

A side observes the peer's close notice and closes its own side.

$RecvFinD(i) \triangleq$
 $\wedge conns[i].fin_d$
 $\wedge conns' = [conns \text{ EXCEPT } ![i].d = \text{IF } @ = \text{"open"} \text{ THEN "closed" ELSE } @,$
 $![i].fin_d = \text{FALSE}]$

$RecvFinA(i) \triangleq$
 $\wedge conns[i].fin_a$
 $\wedge conns' = [conns \text{ EXCEPT } ![i].a = \text{IF } @ = \text{"open"} \text{ THEN "closed" ELSE } @,$
 $![i].fin_a = \text{FALSE}]$

$RecvFin \triangleq \exists i \in Peers : RecvFinD(i) \vee RecvFinA(i)$

A connection both of whose sides are done, with nothing in flight, resets so the dialer may try again.

$SettleConn(i) \triangleq$
 $\wedge conns[i].d = \text{"closed"}$
 $\wedge conns[i].a \in \{\text{"closed"}, \text{"none"}\}$
 $\wedge \neg conns[i].syn \wedge \neg conns[i].fin_d \wedge \neg conns[i].fin_a$
 $\wedge conns' = [conns \text{ EXCEPT } ![i] = NullConn]$

$Settle \triangleq \exists i \in Peers : SettleConn(i)$

$Next \triangleq Dial \vee Accept \vee RecvFin \vee Settle$

Fairness is per connection: each specific incoming open, close notice and settle is eventually processed, and disconnected nodes keep redialling. Existential fairness over all connections would let one peer's endless redial cycle discharge the obligation while the other connection's accept starves — a scheduler artifact, not a policy behaviour. The flap under a symmetric policy contains every one of these actions infinitely often, so it remains a fair counterexample.

$Spec \triangleq$
 $Init$
 $\wedge \Box [Next]_{vars}$
 $\wedge WF_{vars}(Dial)$
 $\wedge \forall i \in Peers :$
 $\wedge WF_{vars}(AcceptConn(i))$
 $\wedge WF_{vars}(RecvFinD(i))$
 $\wedge WF_{vars}(RecvFinA(i))$
 $\wedge WF_{vars}(SettleConn(i))$

A connection fully established on both sides, with nothing else alive.

$Stable \triangleq$

$\exists i \in Peers :$

$\wedge connns[i].d = \text{"open"} \wedge connns[i].a = \text{"open"} \wedge \neg connns[i].syn$
 $\wedge connns[Other(i)] = NullConn$

Liveness: the pair eventually settles on exactly one connection and keeps it. Violated by both symmetric policies (perpetual flap); holds under the asymmetric tie-break.

$EventuallyOneConnection \triangleq \Diamond \Box Stable$

Safety invariants.

$TypeOK \triangleq$

$connns \in [Peers \rightarrow [d : ConnState, a : ConnState,$
 $syn : BOOLEAN, fin_d : BOOLEAN, fin_a : BOOLEAN]]$

No node ever considers two connections to the same peer open at once — every policy enforces the draft’s “at most one” locally. The flap is not a violation of the stated rule; it lives entirely in the gap the rule leaves open.

$AtMostOnePerView \triangleq$

$\forall p \in Peers : Cardinality(ViewOpen(p)) \leq 1$
